

FIRST EDITION · 2026

AI CODING, VIBE CODING & MODERN AI DEVELOPMENT HANDBOOK

A Practical Field Guide to Building Software with AI

WRITTEN BY

Mohammed Anasuddin Zaid

Copyright

© 2026 Mohammed Anasuddin Zaid. All rights reserved.

No part of this handbook may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

This handbook is a personal knowledge compilation created for educational purposes. Tool names, product names, and trademarks mentioned in this handbook belong to their respective owners and are used here only for reference and educational context.

First Edition — 2026

Preface

A few years ago, writing software meant typing every line yourself, one function at a time, one bug at a time. That world hasn't disappeared, but it has been joined by a new one — a world where you can describe what you want in plain English and watch a working prototype take shape in front of you. Some people call this “vibe coding.” Others call it AI-assisted engineering. I call it the biggest shift in how software gets built since the arrival of the modern IDE.

I put this handbook together because I kept noticing the same gap everywhere I looked: plenty of hype about AI tools, very little grounded, structured, practical teaching about how to actually use them well — how to prompt properly, how to review AI-generated code with a critical eye, how to build real agentic systems instead of toy demos, and how to avoid the traps that turn a promising AI-assisted project into unmaintainable spaghetti.

This book is my attempt to close that gap. It is written the way I wish someone had explained this to me — in plain language, with real examples, real code, real mistakes, and real lessons, rather than marketing language dressed up as engineering advice.

Whether you are a student trying to understand what all the fuss around Cursor and Copilot is about, a working developer deciding whether to bring agentic tools into your daily workflow, or a founder trying to ship a product fast without drowning in technical debt — this handbook is for you.

Read it in order if you're new to this space, or jump directly to the tool or topic you need. Every chapter is designed to stand on its own while still building toward a complete, practical understanding of modern AI-powered development.

— Mohammed Anasuddin Zaid

Acknowledgements

This handbook draws on the collective, ongoing work of the global engineering community — the open-source maintainers, the researchers publishing their findings openly, the developers writing honest blog posts about what broke in production, and the countless people sharing prompts, workflows, and hard-earned lessons in public forums and communities.

Thanks are due to everyone building and documenting the tools covered in these pages, and to every developer who takes the time to write down what actually happened when they shipped something to production — those are the notes that make books like this possible.

Finally, thanks to every reader who picks this up hoping to build something real. That intent is what this handbook is written for.

About the Author

I'm Mohammed Anasuddin Zaid (Anas), a 17-year-old software engineer, technology founder, and theoretical physicist operating at the intersection of advanced mathematical logic and scalable product execution. Driven by deep technical curiosity and intense velocity, I build modern software solutions, lead startup operations, and solve complex algorithmic problems.

Currently, I operate as a Technical Product Manager (TPM) and Software Engineer, scaling my own SaaS agency and serving as a C-Suite executive for multiple fast-growing startups. My core technical architecture stack spans HTML/CSS, JavaScript, React, Node.js, Python, and C++, allowing me to seamlessly bridge the gap between high-level business strategy and high-performance, low-level code execution.

As an independent theoretical physicist, I have built a profound analytical foundation by authoring 13 research papers focusing on cosmology, gravitation, and quantum frameworks. Highlighting this track record, my original paper on Spacetime Vortex Gravity and Collapse-Origin Cosmogenesis was officially accepted for presentation at the prestigious international physics seminar (ICAGAC 2026) in Berlin. I treat code the same way I treat physics: using first-principles thinking, mathematical rigor, and strict structural efficiency to engineer elite systems.

Beyond tech and physics, I thrive in high-stakes competitive strategy. I am a former National-Level Chess Player, utilizing tactical calculation and long-term foresight in both board games and corporate decisions. Additionally, I am a highly decorated operator in international simulations, having participated in over 30 Model United Nations (MUN) conferences and winning 20+ top-tier titles, including Best Delegate and Best Player.

On the startup and innovation front, I led the development and presentation of CareSpeak, an assistive communication platform designed to help patients with speech or mobility limitations communicate using hand and eye gestures through an accessible, privacy-first approach. This project secured 5th place nationally at a national-level hackathon contested by 179 teams and nearly 700 participants from across India, including students from IITs and NITs. The project has since been recognized with an incubation opportunity under T-Hub, one of India's leading startup incubators, alongside recognition from Startup India.

Whether I am optimizing a software pipeline, directing product roadmaps, or presenting computational models on an international stage, I focus on intellectual honesty, deep understanding, and absolute execution.

Core Stack & Expertise

Domain	Details
Languages & Frameworks	HTML/CSS, JavaScript (JS), React, Node.js, Python, C++, SQL
Leadership & Strategy	Technical Product Management (TPM), Startup Operations, C-Suite Execution, Agile
Analytical Foundations	Theoretical Physics, Cosmological Research, Strategic Game Theory

Learning Roadmap

This handbook is organized into six parts. You can follow it cover to cover, or use the roadmap below to jump straight to what you need.

Part	Focus	Chapters
Part I	Foundations of AI Coding & Vibe Coding	1-3
Part II	AI Chat & Reasoning Tools	4-6
Part III	AI IDEs & Coding Agents	7-11
Part IV	Agentic AI, RAG, Frameworks & the Wider Tool Ecosystem	12-17
Part V	Infrastructure, Security & Production	18-21
Part VI	Real-World Projects & Career Prep	22-27

Table of Contents

Front Matter Copyright · Preface · Acknowledgements · About the Author · Learning Roadmap · List of Figures · List of Tables · Abbreviations

Part I — Foundations - Chapter 1 — Understanding AI Coding & Vibe Coding - Chapter 2 — Prompt Engineering Fundamentals - Chapter 3 — Setting Up Your AI Development Environment

Part II — AI Chat & Reasoning Tools - Chapter 4 — ChatGPT for Developers - Chapter 5 — Claude for Developers - Chapter 6 — Gemini & Google AI Studio

Part III — AI IDEs & Coding Agents - Chapter 7 — GitHub Copilot - Chapter 8 — Cursor: The AI-Native Code Editor - Chapter 9 — Windsurf and the Broader AI IDE Landscape - Chapter 10 — Autonomous Coding Agents & CLI Tools - Chapter 11 — Model Context Protocol (MCP) & Tool Calling

Part IV — Agentic AI, RAG, Frameworks & the Wider Tool Ecosystem - Chapter 12 — Retrieval-Augmented Generation (RAG) & Vector Databases - Chapter 13 — Agent Frameworks: LangChain, LlamaIndex, CrewAI & AutoGen - Chapter 14 — No-Code & Low-Code AI Builders - Chapter 15 — Running Models Locally: Ollama, LM Studio & Open Models - Chapter 16 — The Extended Tool Ecosystem: Search, Assistants & Automation - Chapter 17 — AI Security, Safety & Guardrails

Part V — Infrastructure, Security & Production - Chapter 18 — Evaluation, Observability & Benchmarking - Chapter 19 — Deployment & DevOps for AI-Powered Applications - Chapter 20 — Fine-Tuning, Evaluation Datasets & Model Customization - Chapter 21 — Vision, Voice & Multimodal AI Development

Part VI — Real-World Projects & Career Prep - Chapter 22 — Project 1: RAG-Powered Support Chatbot - Chapter 23 — Project 2: AI Code Reviewer - Chapter 24 — Project 3: Multi-Agent Research Assistant - Chapter 25 — Project 4: AI SaaS Starter with Billing & Auth - Chapter 26 — Common Mistakes Across the AI Development Lifecycle - Chapter 27 — Career Preparation & The Road Ahead

Back Matter Glossary · References & Further Reading · Appendix: Cheat Sheets · Index

List of Figures

Figure	Description
Fig. 1.1	The AI coding feedback loop (prompt → model → code → review)
Fig. 5.1	Claude Code’s plan → read → diff → test loop
Fig. 10.1	The generic agentic plan → act → observe loop
Fig. 12.1	RAG pipeline: embed, retrieve, augment, generate
Fig. 13.1	Multi-agent role separation (planner / researcher / writer)
Fig. 16.1	Automation pipeline connecting chat, agent, and workflow tools
Fig. 19.1	CI/CD pipeline for an AI-powered feature
Fig. 22.1	Support-bot architecture diagram
Fig. 25.1	SaaS starter architecture with auth, billing, and AI layer

List of Tables

Table	Description
Table 1.1	Evolution of AI coding tools by era
Table 1.2	Advantages vs. limitations of AI coding
Table 2.1	The four pillars of a strong prompt
Table 3.1	Recommended baseline stack for AI-assisted projects
Table 6.1	Google AI Studio strengths and considerations
Table 9.1	Comparison of major AI-native IDEs
Table 10.1	Notable CLI/agentic coding tools
Table 12.1	Core RAG components
Table 13.1	Agent framework comparison
Table 14.1	No-code/low-code AI builder comparison

Table 16.1	Extended tool ecosystem by category
Table 17.1	AI risk categories and mitigations
Table 18.1	Evaluation approaches for AI features
Table 20.1	Fine-tuning approaches compared
Table 21.1	Multimodal AI capability comparison

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command-Line Interface
CI/CD	Continuous Integration / Continuous Deployment
IDE	Integrated Development Environment
LLM	Large Language Model
MCP	Model Context Protocol
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit
SaaS	Software as a Service
UI/UX	User Interface / User Experience

Part I — Foundations of AI Coding & Vibe Coding

Chapter 1

Understanding AI Coding & Vibe Coding

1.1 What Is AI Coding?

AI coding is the practice of using large language models (LLMs) as an active participant in software development — not just autocomplete, but a collaborator that can read a codebase, understand intent, write multi-file changes, run tests, and fix its own mistakes when guided correctly.

Traditional coding tools helped you type faster: syntax highlighting, autocomplete, linters. AI coding tools go further — they can reason about what you’re trying to build, propose an approach, and generate the implementation. The developer’s job shifts from typing every character to directing, reviewing, and validating.

Definition AI Coding: the use of AI models (via chat, IDE integration, or autonomous agents) to generate, modify, review, or reason about source code as part of the software development lifecycle.

1.2 What Is Vibe Coding?

“Vibe coding” is a term that spread quickly through the developer community to describe a specific style of AI-assisted building: you describe the outcome you want in natural language — the “vibe” of the product — and let the AI handle most of the implementation details, iterating conversationally rather than by hand-writing code line by line.

It’s called vibe coding because the developer often isn’t reading every generated line closely at first — they’re steering by outcome (“make the header more premium,” “add a dark mode,” “the login flow feels clunky, smooth it out”) and evaluating the result the way a product designer evaluates a mockup, rather than the way a compiler evaluates syntax.

Important Distinction Vibe coding is excellent for prototypes, MVPs, internal tools, and creative/interactive projects. It becomes risky for production systems handling money, personal data, or safety-critical logic unless it’s paired with real code review, tests, and security checks. Speed without review is how technical debt and vulnerabilities creep in.

1.3 A Short History: From Autocomplete to Autonomous Agents

Table 1.1 — Evolution of AI coding tools by era

Era	Approx. Period	What Changed
Static autocomplete	2000s–2015	IDEs suggested variable/method names from static analysis of the codebase.
Statistical code completion	2016–2020	Tools like TabNine used statistical models trained on code to predict the next few tokens.
LLM-powered completion	2021–2022	GitHub Copilot (built on OpenAI Codex) introduced whole-line and whole-function suggestions from natural language context.
Chat-based coding	2022–2023	ChatGPT and similar tools let developers describe a problem conversationally and get full solutions, explanations, and debugging help.
AI-native IDEs	2023–2024	Cursor, Windsurf, and similar editors rebuilt the IDE itself around AI — multi-file edits, codebase-aware chat, inline diffs.
Autonomous coding agents	2024–2025	Devin-style and CLI agents (Claude Code, Aider, OpenCode) began planning, writing, running, and fixing code across many files with minimal supervision.
Agentic + tool-using systems	2025–2026	Agents gained the ability to call

		external tools (via MCP and similar protocols), browse, manage files, and orchestrate multi-step workflows autonomously.
--	--	--

1.4 Why This Shift Matters

This isn’t just a productivity bump. It changes what a single developer — or a two-person startup — can build in a weekend. It changes what “knowing how to code” even means. And it introduces a new set of skills that didn’t matter as much before: knowing how to prompt precisely, how to review AI output critically, how to structure a codebase so an AI agent can navigate it, and how to catch the specific kinds of mistakes AI models tend to make.

Table 1.2 — Advantages vs. limitations of AI coding

Advantage	Limitation
Dramatically faster prototyping and MVP development	Can produce code that looks correct but has subtle logic errors
Lowers the barrier to entry for non-traditional developers	Can generate insecure patterns (e.g., unsafe SQL, weak auth) if not reviewed
Great for boilerplate, CRUD, glue code, tests	Struggles with deep architectural decisions without strong guidance
Can explain unfamiliar codebases quickly	May silently introduce dependencies or patterns inconsistent with the rest of the project
Frees up developer time for design and product thinking	Over-reliance can erode a developer’s own debugging and fundamentals over time

1.5 Common Mistakes Beginners Make

- Accepting AI-generated code without reading it, then being unable to debug it later.
- Writing vague prompts (“make it better”) and blaming the tool for vague results.
- Letting an agent touch production databases or delete files without a review step.
- Treating every AI suggestion as correct by default, especially around security and edge cases.
- Never learning the underlying language or framework, so review and debugging become impossible.

1.6 A Working Mental Model

Think of an AI coding tool as a very fast, very well-read junior-to-mid-level engineer who has read almost the entire internet, but who has never met you, never seen your specific codebase’s conventions, and has no real stake in the outcome. It will do exactly what you ask — including the parts you didn’t mean to ask for. Your job as the senior partner in this relationship is direction, context, and review.

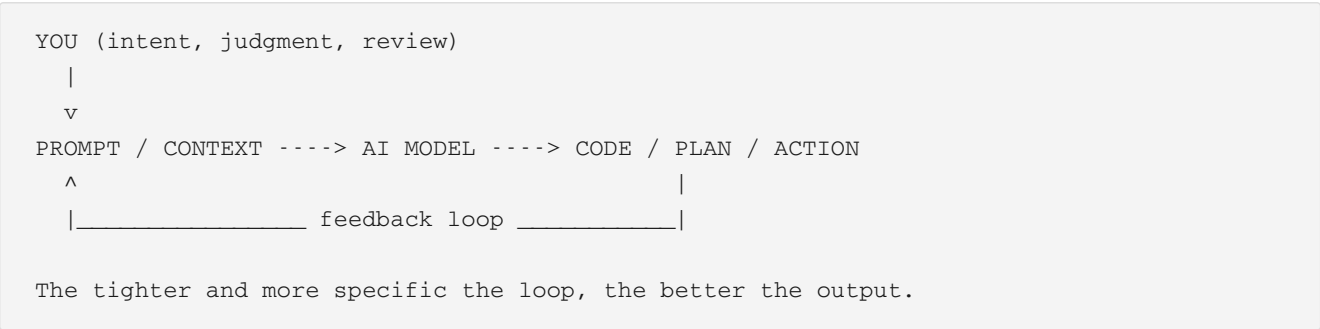


Fig. 1.1 — The AI coding feedback loop.

1.7 Vibe Coding vs. Traditional Coding vs. AI-Assisted Engineering

It helps to separate three things that often get blurred together in conversation:

Mode	Who Drives Decisions	Review Discipline	Best Fit
Traditional coding	Developer, line by line	Continuous, built into writing	Complex, safety-critical, or highly custom systems
AI-assisted engineering	Developer directs, AI implements	Deliberate review of every diff	Day-to-day feature work, refactors, most production code
Vibe coding	AI implements, developer evaluates outcome	Light or deferred, outcome-based	Prototypes, MVPs, internal tools, hackathons

1.8 Chapter Summary

- AI coding is the use of models as active collaborators across the software development lifecycle.
- Vibe coding is a fast, outcome-driven style of AI-assisted building best suited to prototypes and MVPs.
- The field evolved from static autocomplete to fully autonomous, tool-using coding agents in roughly a decade.

- The core skill shift: from typing code to directing, reviewing, and validating AI-generated work.

Interview Questions 1. What is the difference between AI-assisted coding and traditional autocomplete? 2. What are the risks of vibe coding in a production environment? 3. Describe the evolution from Copilot-style completion to autonomous coding agents. 4. Why does review discipline matter more, not less, as AI tools get more capable?

1.9 Exercise

Pick a small personal project idea (a to-do app, a habit tracker, anything simple). Try building the first version in two ways: (1) writing every line yourself for 30 minutes, and (2) describing it conversationally to an AI chat tool for 30 minutes. Compare what you learned, what broke, and how confident you feel in each version’s correctness. Keep notes — you’ll want them again in Chapter 3.

Chapter 2

Prompt Engineering Fundamentals

2.1 What Prompt Engineering Actually Is

Prompt engineering is the practice of structuring your instructions to an AI model so that it reliably produces the output you actually want. It is not a magic trick or a set of secret phrases — it is closer to writing a very clear specification for a very literal, very capable collaborator.

A poorly written prompt leaves room for the model to guess, and models will always guess something rather than ask for clarification unless specifically told to. A well-written prompt removes ambiguity at every point where ambiguity would change the output.

2.2 The Four Pillars of a Strong Prompt

Table 2.1 — The four pillars of a strong prompt

Pillar	What It Means	Example
Context	Background the model needs to understand the task	“This is a Next.js 14 app using the App Router and Tailwind.”

Task	The specific, concrete action you want performed	"Add a paginated table component for the /orders page."
Constraints	Rules, limits, and things to avoid	"Don't use any external UI library; use only Tailwind classes."
Format	How you want the answer structured	"Return only the component code, no explanation."

2.3 Prompting Techniques That Work in Practice

2.3.1 Be Explicit, Not Clever. Models respond better to direct, unambiguous instructions than to hints. Instead of "make this nicer," specify what "nicer" means: spacing, color palette, font weight, animation, whatever you actually mean.

2.3.2 Show, Don't Just Tell (Few-Shot Examples). When you want a specific style of output — a commit message format, a JSON schema, a tone of voice — give one or two examples of exactly what "good" looks like. Models are far more reliable at pattern-matching to an example than at inferring style from an adjective.

2.3.3 Ask for Step-by-Step Reasoning on Hard Problems. For anything involving multi-step logic — debugging, architecture decisions, data transformations — asking the model to reason through the problem before producing a final answer noticeably improves accuracy. This is sometimes called chain-of-thought prompting.

2.3.4 Use Negative Examples. Telling a model what not to do is often as useful as telling it what to do, especially for avoiding overused patterns (e.g., "don't use emoji in commit messages," "don't add try/catch around every single line").

2.3.5 Request Specific Structure. For programmatic use, ask for output in a specific structure — JSON, XML tags, a fixed set of headers — so it can be parsed reliably rather than extracted from free-form prose.

2.3.6 Iterate in Small Steps Rather Than One Giant Prompt. A single 500-word prompt asking for an entire application at once tends to produce shallow, generic results across every part. Breaking the same request into a plan, then implementing one module at a time, produces deeper, more correct output per module.

Cheat Sheet — Prompt Checklist - Have I stated the tech stack / environment? - Have I described the exact outcome, not a vague adjective? - Have I given an example if style/format matters? - Have I stated constraints (what NOT to do)? - Have I specified the output format I need?

2.4 A Real Before / After Example

Weak prompt: “Build me a login page.” *Result:* a generic HTML form with no framework awareness, no styling system, no validation strategy, and no idea which auth provider you use.

Strong prompt:

```
Build a login page for a Next.js 14 (App Router) + TypeScript project.
Use Tailwind for styling, matching our existing design tokens
(bg-slate-950, accent color #FFDE17). Use NextAuth for authentication
with email/password + Google OAuth. Include client-side validation
with react-hook-form and zod. Return only the page.tsx file and
the auth config changes needed, with brief inline comments.
```

The second prompt removes almost every ambiguous decision the model would otherwise have to guess at — framework, styling system, auth library, validation approach, and output scope are all specified.

2.5 Context Engineering: The Next Layer Up

As projects grow, the limiting factor stops being how you phrase a single prompt and becomes what context the model has access to at all. Context engineering is the discipline of deciding what information — files, prior conversation, documentation, examples — gets fed to the model for a given task, and in what order.

Context Source	When to Include It
Relevant source files	Always, for any edit to existing code
Project conventions / style guide	When consistency across the codebase matters
Prior conversation history	When the task builds on earlier decisions
External docs / API references	When using a library the model may not know precisely
Error logs / stack traces	When debugging — paste the real error, not a paraphrase

Common Mistake Dumping an entire large codebase into a prompt “just in case.” This wastes context budget, dilutes attention, and often produces worse results than including only the 2–3 files that are actually relevant to the task.

2.6 Memory Systems in AI Tools

Modern AI coding tools increasingly maintain memory across sessions — remembering

your coding style, your project structure, or past decisions, so you don't have to restate them every time. This ranges from simple project-level config files (like a CLAUDE.md or .cursorrules file) to more sophisticated retrieval-based memory that pulls relevant past context automatically.

Best Practice Maintain a short project-level instructions file (e.g., CLAUDE.md, .cursorrules, or AGENTS.md) describing your stack, conventions, and any hard rules (“never touch the migrations folder without asking”). This single file does more for prompt quality than any clever phrasing.

2.7 Prompting for Different Task Types

Task Type	What to Emphasize in the Prompt
New feature	Context, desired UX, data flow, edge cases
Bug fix	Exact error, reproduction steps, expected vs. actual behavior
Refactor	Scope boundary, what must NOT change (behavior, public API)
Code review	Rubric (bugs, security, style), severity levels, output format
Learning / explanation	Your current understanding, what specifically confuses you

2.8 Chapter Summary

- Good prompts specify context, task, constraints, and format.
- Few-shot examples and step-by-step reasoning requests measurably improve output quality.
- Context engineering — choosing what information the model sees — matters more than clever wording as projects grow.
- A maintained project-instructions file compounds in value over the life of a project.

Interview Questions 1. What are the four pillars of an effective prompt? 2. How does few-shot prompting differ from zero-shot prompting? 3. What is context engineering, and how is it different from prompt engineering? 4. Why can including too much context hurt output quality?

2.9 Exercise

Take a real task from your own work — a bug fix, a small feature, a refactor. Write the laziest possible one-line prompt for it, run it, and note the result. Then rewrite the prompt using all four pillars from Section 2.2, run it again, and compare the two outputs side by side.

Chapter 3

Setting Up Your AI Development Environment

3.1 Choosing Your Starting Stack

Before touching any AI tool, you need a baseline development setup that AI tools can plug into cleanly: a version-controlled project, a consistent formatter/linter, and a clear folder structure. AI tools amplify whatever discipline (or lack of it) already exists in a project — a messy repo produces messier AI output, because the model has less signal about your conventions.

Table 3.1 — Recommended baseline stack for AI-assisted projects

Component	Recommended Baseline	Why It Matters for AI Tools
Version control	Git + GitHub/GitLab	Lets you diff and revert AI-generated changes safely
Formatter	Prettier / Black / gofmt	Keeps AI output visually consistent with the rest of the codebase
Linter	ESLint / Ruff / Pylint	Catches AI mistakes automatically before they're committed
Testing	Jest / Pytest / Vitest	Lets an agent verify its own changes actually work
Project instructions file	CLAUDE.md / .cursorrules / AGENTS.md	Gives every AI tool a consistent source of project context

3.2 The Project Instructions File

Nearly every modern AI coding tool now supports some form of a persistent instructions file that's automatically loaded into context. Names differ by tool, but the purpose is identical: describe your stack, conventions, and hard rules once, and stop

repeating yourself in every prompt.

```
# CLAUDE.md — Project Instructions

## Stack
- Next.js 14 (App Router), TypeScript strict mode
- Tailwind CSS, shadcn/ui components
- PostgreSQL via Prisma ORM

## Conventions
- Use named exports only, no default exports
- All API routes must validate input with zod
- Co-locate tests as *.test.ts next to source files

## Hard Rules
- Never modify files under /prisma/migrations directly
- Never commit .env files
- Always run `npm run lint` before considering a task done
```

Best Practice Keep this file under one page. A short, precise file gets read and followed; a ten-page constitution gets skimmed and ignored — by AI models the same way it would be by a new human hire.

3.3 API Keys, Billing, and Model Access

Most serious AI coding tools require either a subscription (ChatGPT Plus, Claude Pro, GitHub Copilot) or a pay-as-you-go API key (OpenAI, Anthropic, Google). For teams, API-based billing is usually more predictable at scale, while subscriptions are simpler for individual developers. Always set usage limits/budgets where the provider allows it — an unbounded agent loop calling an API repeatedly can generate a surprising bill overnight.

Common Mistake Hardcoding API keys directly into source files. Always use environment variables (.env, loaded via a library like dotenv) and add .env to .gitignore before your first commit — not after.

3.4 Terminal, IDE, or Browser: Where Should You Work?

Environment	Best For	Examples
Browser chat	Exploration, learning, one-off scripts, planning	ChatGPT, Claude.ai, Gemini
AI-native IDE	Full projects, multi-file editing, day-to-day development	Cursor, Windsurf

CLI / terminal agent	Automation, CI integration, headless/agent workflows	Claude Code, Aider, OpenCode
Cloud AI builder	Rapid prototyping, no local setup, sharing instantly	Bolt.new, Lovable, Replit, v0

3.5 A Sensible Folder Structure for AI-Assisted Projects

```
my-project/
├── CLAUDE.md           # project instructions for AI tools
├── .cursorrules        # (if using Cursor)
├── .env.example        # documents required env vars, never real secrets
├── src/
│   ├── components/
│   ├── lib/
│   └── app/
├── tests/
├── docs/
│   └── architecture.md
├── .github/
│   └── workflows/      # CI, including AI-eval tests if applicable
```

3.6 Chapter Summary

- A disciplined baseline (git, linting, tests) makes every AI tool more effective, not less necessary.
- A short, precise project-instructions file compounds in value across a project’s lifetime.
- Choose your working environment (browser, IDE, CLI, cloud builder) based on the task, not habit.
- Treat API keys and usage budgets as a security and cost concern from day one.

3.7 Exercise

Add a CLAUDE.md (or equivalent) file to a real project you’re working on. Include your stack, three conventions you actually follow, and at least one hard rule. Use it for a week and note whether AI suggestions become noticeably more consistent with your codebase.

Part II — AI Chat & Reasoning Tools

Chapter 4

ChatGPT for Developers

4.1 Overview

ChatGPT, built on OpenAI’s GPT model family, was the tool that introduced most developers to conversational AI assistance. Its strengths for coding work lie in broad general knowledge, strong explanation ability, and a mature plugin/tool ecosystem including code execution (Code Interpreter / Advanced Data Analysis) and custom GPTs.

4.2 Where ChatGPT Fits in a Developer Workflow

Use Case	Fit
Explaining unfamiliar code or errors	Excellent — clear, structured explanations
One-off scripts and data analysis	Excellent — especially with code execution enabled
Full multi-file project generation	Moderate — better suited to IDE-integrated tools for large codebases
Architecture discussion / brainstorming	Excellent — strong at exploring trade-offs conversationally
Long-running autonomous coding tasks	Weak compared to dedicated coding agents

4.3 Practical Workflow: Debugging With ChatGPT

1. Paste the full error message and stack trace verbatim — never paraphrase an error.
2. Include the relevant function or file, not just the error line.
3. State what you expected to happen versus what actually happened.
4. Ask it to list possible causes ranked by likelihood before jumping to a fix.
5. Apply one fix at a time and re-test — don’t accept a bundle of five changes at once.

Definition Code Interpreter (Advanced Data Analysis): a sandboxed Python execution environment inside ChatGPT that lets the model actually run code, process files, and return real output rather than just guessing what code would do.

4.4 Custom GPTs and Automation

Custom GPTs allow developers to package a specific persona, instruction set, and set of tools (via Actions, which call external APIs) into a reusable configuration — effectively a lightweight, shareable agent for a narrow task, such as reviewing pull requests against a specific style guide or answering questions from a fixed knowledge base.

4.5 Limitations to Keep in Mind

- No native awareness of your local codebase unless you paste files in manually.
- Long conversations can lose earlier context, causing inconsistent suggestions.
- Like all LLMs, it can generate plausible-looking but incorrect library APIs — always verify against real documentation.

4.6 Chapter Summary

- ChatGPT excels at explanation, brainstorming, and self-contained scripts.
- It is not codebase-aware by default — pair it with an IDE tool for large projects.
- Always verify generated library/API usage against real documentation.

Chapter 5

Claude for Developers

5.1 Overview

Claude, built by Anthropic, is known among developers for long-context handling, careful step-by-step reasoning, and strong performance on large refactors and multi-file understanding. Claude Code, Anthropic's terminal-based agent, extended this into an autonomous coding tool that can read a repository, plan a change, implement it across files, run tests, and iterate.

5.2 Claude Code: Agentic Development From the Terminal

Unlike a chat window, Claude Code operates directly against your file system from the

command line. It can read files, propose diffs, run shell commands (with permission), and loop until tests pass. This makes it suited to tasks like large refactors, dependency upgrades, and repetitive multi-file changes that would be tedious to do by hand or through a chat interface.

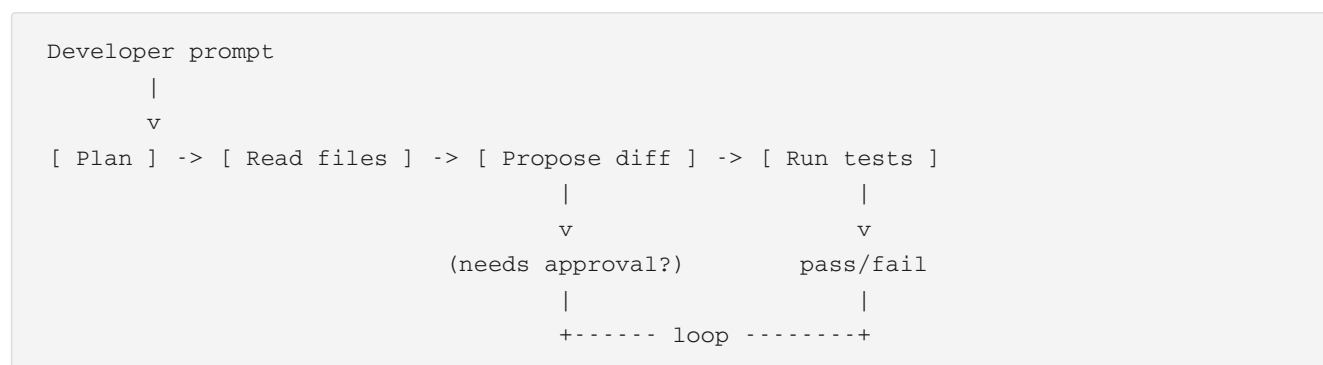


Fig. 5.1 — Claude Code’s plan → read → diff → test loop.

5.3 Practical Workflow: A Multi-File Refactor

1. Describe the refactor goal precisely (e.g., “migrate all class components to functional components with hooks”).
2. Point to the exact folder scope — don’t let the agent roam the entire repo unscoped.
3. Ask for a plan first, review it, then approve execution.
4. Require tests to pass before considering the task complete.
5. Review the diff yourself before merging — an agent finishing without errors is not the same as an agent finishing correctly.

Best Practice For any agentic tool with file-system or shell access, work inside a git branch with clean working state so every change is easy to diff and revert.

5.4 Claude Cowork and Broader Access Points

Beyond the terminal, Claude is also reachable through a chat interface (claude.ai and the mobile app), through an API for custom integrations, and through agentic surfaces built for non-developer knowledge work. The consistent thread across all of them is the same reasoning model — what changes is how much file-system and tool access it is given, and how much of the workflow is automated versus reviewed by a human at each step.

5.5 Chapter Summary

- Claude is well suited to large-context tasks: big refactors, whole-codebase understanding, careful reasoning.
- Claude Code brings agentic, file-system-aware development to the terminal.
- Scope, plan-then-execute, and test-gating are the three habits that keep agentic tools safe.

Chapter 6

Gemini & Google AI Studio

6.1 Overview

Gemini is Google’s model family, accessible through the Gemini app, the Gemini API, and Google AI Studio — a browser-based playground for prompting, comparing model versions, and prototyping with function calling before writing any production code.

6.2 Google AI Studio Workflow

1. Prototype a prompt in AI Studio’s playground, adjusting temperature and system instructions live.
2. Test function-calling/tool definitions directly in the UI before wiring them into your app.
3. Export the working configuration as ready-to-use API code (Python, Node.js, or REST).
4. Move to production using the Gemini API with proper key management.

Table 6.1 — Google AI Studio strengths and considerations

Strength	Consideration
Very large context windows, useful for whole-repo or long-document tasks	Ecosystem tooling for coding agents is less mature than Claude Code / Cursor as of writing
Tight integration with Google Cloud and Workspace data	Best value often tied to being inside the Google ecosystem
Strong multimodal (text + image + video) understanding	Rapid version churn — always check current model names before committing to one

6.3 Practical Use Cases

- Summarizing and reasoning over very long documents (contracts, research papers, entire codebases pasted as text).
- Prototyping multimodal features — a prompt that takes a screenshot and returns a bug diagnosis, for example.
- Rapid function-calling prototypes before wiring the same schema into a production backend.

6.4 Chapter Summary

- Google AI Studio is a fast way to prototype prompts and tool-calling before writing app code.
- Gemini's large context windows suit long-document and whole-repo comprehension tasks.
- Always verify current model names — this is one of the fastest-moving parts of the ecosystem.

Part III — AI IDEs & Coding Agents

Chapter 7

GitHub Copilot

7.1 Overview

GitHub Copilot was the tool that brought AI code completion into the mainstream IDE experience. It integrates directly into VS Code, JetBrains IDEs, and Neovim, offering inline completions, a chat panel, and — in its newer “agent mode” — the ability to make multi-file changes and open pull requests autonomously.

7.2 Copilot Features at a Glance

Feature	What It Does
Inline completion	Suggests the next line(s) of code as you type
Copilot Chat	Conversational panel for explaining, refactoring, and generating code in context
Copilot Edits / Agent Mode	Plans and applies multi-file changes across a workspace
Copilot for Pull Requests	Generates PR descriptions and reviews diffs automatically
Copilot CLI	Suggests and explains shell commands directly in the terminal

7.3 Getting the Most Out of Inline Completion

- Write a clear function name and docstring/comment first — completion quality tracks the clarity of the signature.
- Keep related files open in your editor; Copilot uses open tabs as additional context.
- Reject bad suggestions quickly rather than editing around them — a clean reject keeps future suggestions on track.

7.4 Copilot in Enterprise Settings

Organizations adopting Copilot at scale typically care about three additional things beyond raw suggestion quality: whether suggestions are filtered against public code matches (to reduce licensing risk), whether an admin can enforce policy across the organization, and how usage is audited for compliance purposes. These enterprise controls are often what determines adoption more than the completion quality itself.

7.5 Chapter Summary

- Copilot remains the most widely adopted in-editor completion tool, with growing agentic capability.
- Clear function signatures and open related tabs meaningfully improve suggestion quality.
- Agent mode and PR review features move Copilot beyond simple autocomplete into light agentic territory.

Chapter 8

Cursor — The AI-Native Code Editor

8.1 Overview

Cursor is a fork of VS Code rebuilt around AI as a first-class citizen rather than a bolted-on extension. Its core differentiators are codebase-aware chat (it indexes your whole project), multi-file “Composer” edits, and inline diff review that lets you accept or reject AI changes line by line.

8.2 Core Workflows

Workflow	Concept	Best For
Inline edit	Select code, describe the change	Small, localized edits
Chat (codebase-aware)	Ask questions referencing @files or @codebase	Understanding unfamiliar code, planning
Composer / Agent mode	Describe a feature, let it plan + edit multiple files	New features spanning several files
Terminal integration	Ask Cursor to run and interpret command output	Debugging build/test failures

8.3 The .cursorrules File

Cursor reads a project-level rules file automatically on every request, making it the natural home for the conventions discussed in Chapter 3 — stack details, style rules, and hard constraints the AI should never violate.

Common Mistake Accepting large Composer diffs wholesale without reading them. Cursor makes acceptance a single click, which is exactly why deliberate review discipline matters more here, not less.

8.4 A Realistic Cursor Session

A typical, well-run Cursor session for adding a feature looks like this: open the relevant files, reference them explicitly in chat with `@filename`, ask Composer to draft a plan before touching any code, read the plan and correct any wrong assumptions, approve execution, review the diff file by file, run the test suite, and only then commit. Skipping the plan-review step is the single most common way Cursor sessions go off the rails on non-trivial features.

8.5 Chapter Summary

- Cursor’s core advantage is deep codebase awareness baked into the editor itself.
- Composer/Agent mode enables multi-file feature development from a single natural-language prompt.
- Low-friction acceptance of AI diffs raises the importance of deliberate, line-by-line review.

Chapter 9

Windsurf and the Broader AI IDE Landscape

9.1 Windsurf Overview

Windsurf (by Codeium) is another AI-native editor competing directly with Cursor. Its signature feature, “Cascade,” is an agentic flow that can autonomously plan and execute multi-step coding tasks while showing its reasoning and the commands it intends to run before running them.

9.2 Comparing the AI IDE Field

Table 9.1 — Comparison of major AI-native IDEs

Tool	Signature Feature	Notable Strength
Cursor	Composer multi-file agent	Mature ecosystem, deep VS Code compatibility
Windsurf	Cascade agentic flow	Transparent step-by-step execution trace
Continue.dev	Open-source, model-agnostic	Bring-your-own-model flexibility, self-hostable
Cline (VS Code ext.)	Human-in-the-loop agent	Explicit approval before every file/command action
Roo Code	Community-driven Cline fork	Rapid feature experimentation from an active fork community

9.3 Continue.dev in Depth

Continue.dev is an open-source extension for VS Code and JetBrains that lets you plug in any model — a hosted API, a local Ollama model, or an enterprise-hosted endpoint — behind a consistent chat and autocomplete interface. It appeals to teams who want to standardize a workflow without being locked to a single model provider, and to developers who want full control over what data leaves their machine.

9.4 Cline and Roo Code in Depth

Cline (formerly Claude Dev) popularized a strict human-in-the-loop agent pattern inside VS Code: before every file edit or terminal command, it shows exactly what it intends to do and waits for explicit approval. This trades some speed for a very high degree of control, which makes it a common choice for developers working in sensitive codebases. Roo Code is an actively maintained community fork of Cline that iterates quickly on the same core idea, often shipping experimental features (custom modes, browser automation, checkpoints) ahead of the upstream project.

9.5 Choosing Between Them

For most developers, the honest answer is: try two or three on a real (not toy) task for a week each. Benchmarks change monthly in this space, and the “best” tool is often the one whose review/approval flow matches your own risk tolerance — some developers want full autonomy with review after the fact, others want approval before every single file change.

9.6 Chapter Summary

- Windsurf’s Cascade and Cursor’s Composer solve the same problem with different transparency trade-offs.
- Open-source options like Continue.dev and Cline suit developers who want model flexibility or explicit control.
- Tool choice is often a fit question (approval style, ecosystem) more than a strict capability ranking.

Chapter 10

Autonomous Coding Agents & CLI Tools

10.1 What Makes a Tool “Agentic”

An agentic coding tool doesn’t just respond to a single prompt — it plans a sequence of actions, executes them (reading files, writing code, running commands), observes the result, and adjusts its plan, repeating this loop until the task is done or it needs your input. This loop is what separates Aider, Claude Code, and OpenCode from a simple chat window.

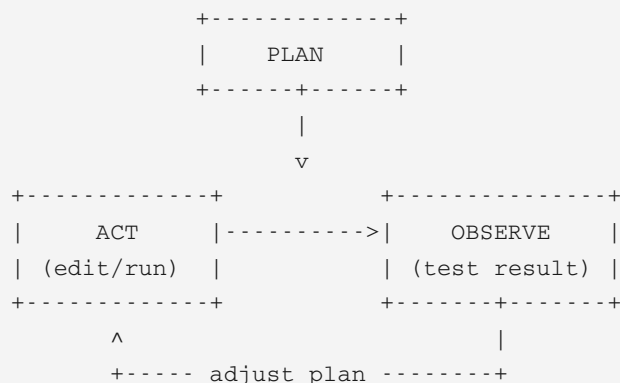


Fig. 10.1 — The generic agentic plan → act → observe loop.

10.2 Notable CLI/Agentic Tools

Table 10.1 — Notable CLI/agentic coding tools

Tool	Interface	Notable Trait
Claude Code	Terminal	Deep repo awareness, permission-gated shell access
Aider	Terminal	Git-native — every change is a real commit, easy to

		review/revert
OpenCode	Terminal	Open-source, model-agnostic agent runner
Amazon Q Developer	IDE + CLI	Deep AWS service awareness for cloud-native projects
Sourcegraph Cody	IDE	Strong code search across very large enterprise codebases

10.3 Aider in Depth

Aider treats every AI-generated change as a real, atomic git commit with a generated commit message — nothing is applied silently outside of version control. This makes it especially easy to review history after a session, `git revert` a bad change, or bisect a regression back to the exact AI-generated commit that introduced it. It works well for developers who want an agent that feels like “a very fast pair programmer who commits constantly,” rather than one that produces a single large uncommitted diff.

10.4 Amazon Q Developer and Sourcegraph Cody

Amazon Q Developer is tuned for teams already deep in the AWS ecosystem — it understands AWS service configuration, can review infrastructure-as-code, and integrates with CodeWhisperer-style completions inside supported IDEs. Sourcegraph Cody is built around Sourcegraph’s code search engine, which makes it especially strong at answering “where in this million-line codebase does X happen” — a question general-purpose chat tools struggle with once a codebase exceeds what fits in a context window.

10.5 Safety Practices for Agentic Tools

1. Always run agents inside a version-controlled branch, never directly on main/production.
2. Restrict shell/file permissions to the project directory — never grant broad system access.
3. Require a test pass (or explicit human review) before merging agent-generated changes.
4. Set a hard budget or step limit for long-running autonomous loops to avoid runaway API costs.
5. Never let an agent handle credentials, secrets, or production database access unsupervised.

Important An agent reporting “task complete” is a self-assessment, not a guarantee. Treat it the same way you’d treat a junior engineer’s own claim that their PR is ready — verify before trusting.

10.6 Chapter Summary

- Agentic tools run a plan → act → observe loop rather than answering a single prompt.
- Git-native tools like Aider make every agent action naturally reviewable and revertible.
- Scoped permissions, test-gating, and budget limits are non-negotiable safety practices.

Chapter 11

Model Context Protocol (MCP) & Tool Calling

11.1 What Problem MCP Solves

Before MCP, every AI tool that wanted to connect to Slack, GitHub, or a database needed a custom, one-off integration. The Model Context Protocol standardizes how an AI application discovers and calls external tools and data sources, so a single MCP server (say, for GitHub) can be used by any MCP-compatible client without rewriting the integration each time.

11.2 Core Concepts

Concept	Description
MCP Server	Exposes a set of tools/resources (e.g., “create_issue”, “search_files”) over a standard protocol
MCP Client	The AI application (an IDE, an agent, a chat app) that discovers and calls those tools
Tool	A single callable capability with a defined input/output schema
Resource	Read-only data the server exposes (a file, a document, a database row)

11.3 Function Calling vs. Tool Calling vs. MCP

These terms overlap heavily. “Function calling” and “tool calling” typically refer to the model’s ability to emit a structured request to invoke a specific function with specific arguments, which your application code then executes. MCP standardizes how those tools are described and discovered across different applications, rather than defining the calling mechanism itself — think of it as a common language for exposing tools, on top of which function/tool calling still happens.

```
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "input_schema": {
    "type": "object",
    "properties": { "city": { "type": "string" } },
    "required": ["city"]
  }
}
```

11.4 Building a Minimal MCP Server (Conceptual)

```
# server.py — a minimal MCP-style tool server (conceptual)
from mcp.server import Server
from mcp.types import Tool

app = Server("project-tools")

@app.tool()
def search_files(query: str) -> list[str]:
    """Search the project for files matching the query."""
    return [f for f in list_project_files() if query in f]

@app.tool()
def run_tests(path: str) -> dict:
    """Run the test suite for a given path and return pass/fail counts."""
    result = execute_test_command(path)
    return {"passed": result.passed, "failed": result.failed}
```

Each tool is defined once, with a clear docstring and schema, and can then be reused by any MCP-compatible client — an IDE, a CLI agent, or a custom application — without rewriting the integration.

11.5 Chapter Summary

- MCP standardizes how AI applications discover and call external tools and data sources.
- Function/tool calling is the underlying mechanism; MCP is the shared protocol

layer on top.

- Adopting MCP lets one integration (e.g., a GitHub MCP server) serve many different AI clients.

Interview Questions 1. What problem does MCP solve that plain function calling does not? 2. Explain the difference between an MCP server, an MCP client, a tool, and a resource. 3. Why does standardizing tool discovery matter for teams building multiple AI applications?

Part IV — Agentic AI, RAG, Frameworks & the Wider Tool Ecosystem

Chapter 12

Retrieval-Augmented Generation (RAG) & Vector Databases

12.1 Why RAG Exists

A model’s training data is frozen at a point in time and can’t include your private documents, your company’s internal wiki, or yesterday’s data. RAG solves this by retrieving relevant chunks of external information at query time and inserting them into the prompt, so the model reasons over live, specific data instead of relying solely on what it memorized during training.

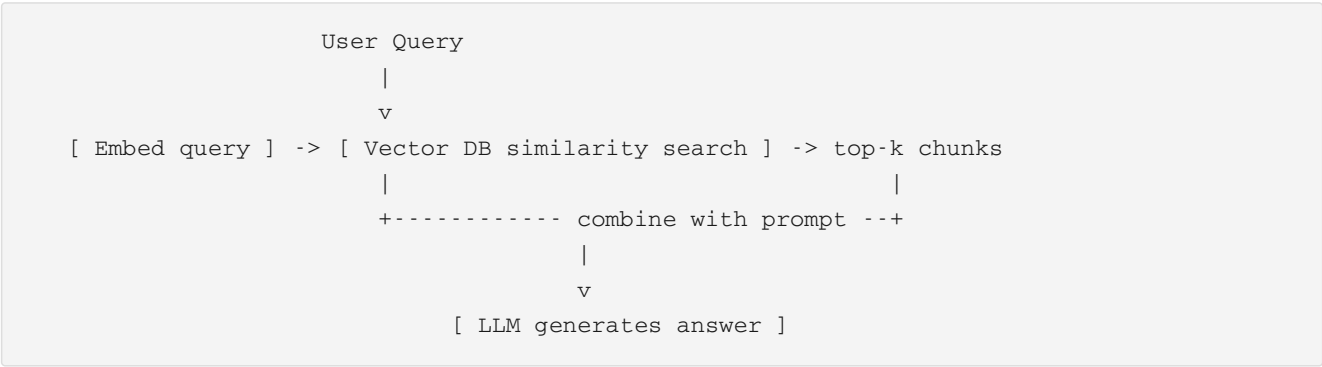


Fig. 12.1 — RAG pipeline: embed, retrieve, augment, generate.

12.2 Core Components

Table 12.1 — Core RAG components

Component	Role	Examples
Embedding model	Converts text into a numeric vector capturing semantic meaning	OpenAI text-embedding-3, Voyage AI, Cohere Embed
Vector database	Stores vectors and performs fast similarity search	Pinecone, Weaviate, Qdrant, pgvector, Chroma
Chunking strategy	Splits documents into retrievable pieces	Fixed-size, semantic, recursive splitting

Orchestration layer	Wires retrieval + generation together	LangChain, LlamaIndex, custom code
---------------------	---------------------------------------	------------------------------------

12.3 Chunking Strategies in Practice

Strategy	How It Works	Trade-off
Fixed-size chunking	Split every N tokens/characters, often with overlap	Simple and fast, but can cut sentences or ideas mid-thought
Recursive chunking	Split on paragraph, then sentence, then word boundaries as needed	Respects natural text structure better than fixed-size
Semantic chunking	Split at points where meaning shifts, using embeddings to detect boundaries	Higher quality retrieval, more compute cost to prepare
Document-aware chunking	Use structure (headings, sections) as natural chunk boundaries	Works very well for structured docs like manuals or specs

12.4 Common Mistakes in RAG Systems

- Chunking documents too large — retrieval returns bloated, unfocused context.
- Chunking too small — retrieval loses surrounding context needed to answer correctly.
- Ignoring metadata filtering (date, source, permissions), leading to irrelevant or unauthorized results.
- Never evaluating retrieval quality separately from generation quality, making it hard to diagnose bad answers.

12.5 Chapter Summary

- RAG grounds model answers in live, external, or private data instead of frozen training data.
- Embedding models + vector databases + chunking strategy are the three levers that determine retrieval quality.
- Evaluate retrieval and generation quality separately to diagnose failures accurately.

Chapter 13

Agent Frameworks: LangChain, LlamaIndex, CrewAI & AutoGen

13.1 Why Frameworks Exist At All

You can call an LLM API directly with a few lines of code, so why use a framework? Because real applications need memory management, retry logic, tool orchestration, multi-agent coordination, and prompt templating — and frameworks package these repeated patterns so you’re not reinventing them for every project.

13.2 Framework Comparison

Table 13.1 — Agent framework comparison

Framework	Primary Focus	Best For
LangChain	General-purpose LLM app orchestration (chains, agents, memory)	Broad, flexible LLM applications with many moving parts
LlamaIndex	Data indexing and retrieval for RAG	Document-heavy RAG applications
CrewAI	Role-based multi-agent collaboration	Simulating a “team” of specialized agents on one task
AutoGen	Conversational multi-agent orchestration	Research and complex agent-to-agent dialogue patterns
Semantic Kernel	Enterprise .NET/Python AI orchestration	Teams already invested in the Microsoft stack
Flowise	Visual, node-based LLM app builder	Prototyping chains and agents without writing orchestration code

13.3 A Minimal Multi-Agent Example (Conceptual)

```
# Conceptual CrewAI-style setup
researcher = Agent(role='Researcher', goal='Gather facts on the topic')
writer = Agent(role='Writer', goal='Turn research into a clear draft')
reviewer = Agent(role='Reviewer', goal='Check accuracy and tone')

crew = Crew(agents=[researcher, writer, reviewer],
            tasks=[research_task, write_task, review_task])
result = crew.kickoff()
```

13.4 LangChain and LlamaIndex in Practice

LangChain’s core abstractions — chains, agents, memory, and tool integrations — make it well suited to applications where an LLM call is one step in a larger pipeline involving multiple data sources and conditional logic. LlamaIndex focuses more narrowly and deeply on the “data” side of that pipeline: ingesting, indexing, and retrieving from documents, databases, and APIs, which is why many teams pair LlamaIndex for retrieval with LangChain (or plain code) for the surrounding orchestration.

13.5 Semantic Kernel and Flowise

Semantic Kernel is Microsoft’s orchestration SDK, popular with teams building on Azure OpenAI and the broader .NET ecosystem; it emphasizes “plugins” and “planners” as its core abstractions, conceptually similar to LangChain’s chains and agents. Flowise takes a different approach entirely — a visual, drag-and-drop canvas for wiring together LLM calls, retrievers, and tools, which makes it a fast way for both technical and non-technical team members to prototype an agent flow before committing to hand-written code.

Common Mistake Reaching for a multi-agent framework before a single well-prompted agent has been tried. Multi-agent systems add coordination overhead and failure modes of their own — use them when a task genuinely benefits from role specialization, not by default.

13.6 Chapter Summary

- Frameworks package memory, orchestration, and tool-calling patterns you’d otherwise rebuild repeatedly.
- LangChain and LlamaIndex suit general orchestration and RAG respectively; CrewAI and AutoGen suit multi-agent collaboration.
- Start with the simplest architecture (single agent, direct API calls) and add framework complexity only when justified.

Chapter 14

No-Code & Low-Code AI Builders

14.1 Overview

A parallel track to AI-native IDEs has grown around no-code/low-code builders that let non-engineers (and time-pressed engineers) go from a prompt to a deployed web app in minutes. Bolt.new, Lovable, Replit, v0, Firebase Studio, and similar tools trade some flexibility for dramatically faster iteration on the first version of a product.

Table 14.1 — No-code/low-code AI builder comparison

Tool	Primary Output	Strength
Bolt.new	Full-stack web apps in-browser	Fast, complete scaffolding including backend
Lovable	Full-stack web apps with design focus	Polished UI output from natural-language prompts
v0 (Vercel)	React/Next.js UI components	High-quality component-level generation, easy to export into a real codebase
Replit	General-purpose cloud IDE + AI agent	Instant hosting and deployment alongside AI generation
Firebase Studio	Full-stack apps on Google’s Firebase backend	Tight integration with Firebase auth, database, and hosting
Manus	General-purpose autonomous task agent	Long-running, multi-step tasks beyond just coding
Emergent	Full-stack app generation from a prompt	Rapid MVP scaffolding aimed at founders

14.2 When to Use a No-Code Builder vs. a Real Codebase

Situation	Recommendation
Validating an idea, pitching a concept	No-code builder — speed matters more than architecture
Internal tool with a small, known user base	No-code builder is often fine long-term
Product with real users, compliance needs, or complex logic	Export/migrate to a real codebase with proper engineering practices
Team of multiple engineers collaborating long-term	Real codebase — no-code tools rarely scale well with team size

14.3 Migrating Out of a No-Code Builder

Most no-code builders eventually need to hand off to a real, version-controlled codebase once a product gains real users or compliance requirements. A clean migration typically follows this order: export the generated code, set up proper git version control immediately, add a real test suite before making any further changes, then incrementally replace generated glue code with reviewed, understood implementations — starting with anything touching authentication, payments, or personal data.

Best Practice Never let a no-code-generated authentication or payment integration go untouched into production without a proper security review, even if it “looks like it works.” These are exactly the areas where generated glue code hides the riskiest assumptions.

14.4 Chapter Summary

- No-code AI builders trade some flexibility for dramatically faster time-to-first-version.
- They excel at validation and internal tools, and typically need migration once a product has real production requirements.
- Choosing the right tool is a question of project stage, not just personal preference.

Chapter 15

Running Models Locally: Ollama, LM Studio & Open Models

15.1 Why Run Models Locally

Cloud APIs are convenient, but local models matter for privacy-sensitive data, offline development, cost control at high volume, and experimentation without per-token billing. Open-weight model families like Llama, Qwen, DeepSeek, and Mistral can now run on consumer hardware with tools like Ollama and LM Studio.

Tool	Interface	Best For
Ollama	CLI + local API server	Developers who want scriptable, terminal-first local models
LM Studio	Desktop GUI	Trying models interactively without touching a terminal

Hugging Face (Transformers)	Python library + model hub	Fine-tuning, research, and custom pipelines
OpenRouter	Hosted API gateway to many models	Comparing/switching between providers without separate integrations

15.2 A Minimal Ollama Workflow

```
# Pull and run a model locally
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder:7b

# Or call it via the local API from your own code
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5-coder:7b",
  "prompt": "Write a Python function to reverse a linked list"
}'
```

Best Practice Match model size to your hardware honestly. A 7B model on a laptop with 16GB RAM will run comfortably; a 70B model will not, regardless of how good the benchmarks look on paper.

15.3 Open Model Families Worth Knowing

Family	Known For
Llama (Meta)	Broad general-purpose open-weight models, large ecosystem support
Qwen (Alibaba)	Strong multilingual and coding-focused variants
DeepSeek	Strong reasoning and coding performance relative to model size
Mistral	Efficient, permissively licensed smaller models

15.4 Hugging Face and ComfyUI

Hugging Face functions as both a model hub (for downloading open-weight models and datasets) and a Python library ecosystem (`transformers` , `diffusers`) for loading, fine-tuning, and running those models directly in code. ComfyUI takes a different, node-based visual approach specifically for image and video generation workflows — chaining models, samplers, and post-processing steps on a visual canvas, popular with teams building custom generative-media pipelines rather than plain text applications.

15.5 Chapter Summary

- Local models trade some capability for privacy, offline access, and predictable cost.
- Ollama and LM Studio are the two most common entry points, CLI-first and GUI-first respectively.
- Choose model size based on your actual hardware, not benchmark leaderboards alone.

Chapter 16

The Extended Tool Ecosystem: Search, Assistants & Automation

16.1 Overview

Beyond chat models and coding agents, a wide layer of AI-powered tools now sits across search, personal productivity, and workflow automation. Understanding where each fits prevents reaching for the wrong tool for a task.

Table 16.1 — Extended tool ecosystem by category

Category	Tools	What They’re For
AI-powered search & research	Perplexity, Phind	Grounded, cited answers to research questions; Phind is developer-question-focused
Terminal & OS-level assistants	Warp AI, Raycast AI	AI-assisted command suggestions and OS-level quick actions
Workflow automation	n8n, Flowise	Connecting APIs and LLM steps into automated pipelines without heavy custom code
Model access gateways	OpenRouter	Single API surface to switch between many hosted model providers

16.2 Perplexity and Phind

Perplexity positions itself as an answer engine rather than a search engine — it retrieves live web sources and synthesizes a cited answer, which makes it useful for

research tasks where you want to verify claims against real sources rather than a model's memorized training data. Phind targets developers specifically, tuning its retrieval and synthesis toward programming documentation, Stack Overflow-style questions, and framework-specific errors.

16.3 Warp AI and Raycast AI

Warp is a terminal built from the ground up with AI assistance baked in — suggesting commands, explaining cryptic error output, and letting you describe an intended shell operation in plain English before it's translated into an actual command. Raycast AI brings similar natural-language convenience to the broader desktop: quick AI actions (summarizing clipboard text, drafting a message, transforming selected text) triggered from a system-wide launcher rather than a dedicated chat window.

16.4 n8n and Workflow Automation

n8n is a self-hostable, node-based automation platform (similar in spirit to Zapier) that has added first-class LLM nodes — meaning a workflow can watch for a new support ticket, embed and search a knowledge base, generate a draft reply with an LLM, and post it to a helpdesk tool, all without writing a dedicated backend service. This category matters because a large share of “AI features” a business actually needs are workflow automation problems, not custom application-development problems.

```
[ New Ticket Trigger ] -> [ Embed + Retrieve KB ] -> [ LLM Draft Reply ] -> [ Post to Helpdesk ]
```

16.5 OpenRouter and Model Gateways

OpenRouter and similar gateways expose a single, OpenAI-compatible API surface that routes to many different underlying providers and models — useful for comparing model quality/cost trade-offs without rewriting integration code for each provider, and for building fallback logic (if one provider is down or rate-limited, route to another automatically).

16.6 Chapter Summary

- Perplexity and Phind specialize AI-powered search for general research and developer questions respectively.
- Warp AI and Raycast AI bring natural-language assistance into the terminal and the desktop shell.
- n8n and similar automation tools solve a large share of practical “AI feature” requests without custom backend code.

- Model gateways like OpenRouter simplify comparing and switching between model providers.

Chapter 17

AI Security, Safety & Guardrails

17.1 New Attack Surfaces Introduced by AI

Adding an LLM to your application introduces failure modes that don't exist in traditional software: prompt injection, data leakage through model output, and over-permissioned tool access that an attacker can manipulate the model into misusing.

Table 17.1 — AI risk categories and mitigations

Risk	Description	Mitigation
Prompt injection	Untrusted input (a document, a webpage) contains hidden instructions that hijack the model's behavior	Treat all retrieved/external content as data, never as instructions; sanitize and clearly delimit it
Data leakage	Model output accidentally reveals sensitive context from the prompt or training	Minimize sensitive data in prompts; redact before sending to third-party APIs
Over-permissioned tools	An agent with broad file/shell/API access can be manipulated into destructive actions	Principle of least privilege — scope every tool grant to only what's needed
Hallucinated dependencies	Model suggests a package name that doesn't exist, which an attacker can register and poison	Always verify package names against the real registry before installing

Security Note Never let an AI agent execute shell commands with unrestricted sudo/root access, and never expose an agent's tool-calling interface directly to untrusted end users without strict validation on every tool call.

17.2 Guardrail Patterns

- **Input validation:** reject or sanitize prompts matching known injection patterns

before they reach the model.

- **Output filtering:** scan model responses for secrets, PII, or disallowed content before returning them to users.
- **Human-in-the-loop checkpoints:** require explicit approval for high-risk actions (payments, deletions, external emails).
- **Rate limiting and budget caps:** prevent runaway loops or abuse from draining API budgets.

17.3 A Layered Guardrail Architecture

```
Untrusted Input
  |
  v
[ Input validation / sanitization ]
  |
  v
[ LLM with scoped tool access ]
  |
  v
[ Output filtering (PII, secrets, policy) ]
  |
  v
[ Human approval for high-risk actions ]
  |
  v
Final Action
```

17.4 Chapter Summary

- AI systems introduce new attack surfaces: prompt injection, data leakage, over-permissioned tools, hallucinated dependencies.
- Treat all external content fed to a model as untrusted data, never as instructions.
- Guardrails — input validation, output filtering, human checkpoints, rate limits — should be designed in from the start, not bolted on after an incident.

Interview Questions 1. What is prompt injection, and how is it different from a traditional injection attack (e.g., SQL injection)? 2. Why is “least privilege” especially important for agentic tools with shell or file access? 3. Describe a layered guardrail architecture for a customer-facing AI chatbot.

Part V — Infrastructure, Security & Production

Chapter 18

Evaluation, Observability & Benchmarking

18.1 Why “It Seems to Work” Isn’t Enough

LLM-powered features fail silently far more often than traditional code — a wrong answer often looks just as fluent and confident as a correct one. Without structured evaluation, teams discover regressions from user complaints instead of from their own testing.

18.2 Core Evaluation Approaches

Table 18.1 — Evaluation approaches for AI features

Approach	Description	When to Use
Golden dataset testing	Run a fixed set of known input/expected-output pairs on every change	Regression testing before every deployment
LLM-as-judge	Use a separate model call to score outputs against a rubric	Scaling evaluation beyond what humans can review manually
Human review sampling	Manually review a random sample of real production outputs	Catching issues automated metrics miss
A/B testing	Compare two prompts/models on real traffic with measured outcomes	Validating a specific change’s real-world impact

18.3 Observability in Production

Once deployed, an AI feature needs the same operational visibility as any other production system: logging every prompt/response pair (with appropriate privacy handling), tracking latency and token cost per request, and alerting on anomalies like

sudden spikes in refusals or errors.

Best Practice Log structured metadata (model version, prompt template version, latency, token counts) alongside every request from day one. Retrofitting observability after a production incident is far more painful than building it in from the start.

18.4 Building a Golden Dataset

A golden dataset is a curated set of representative input/output pairs that reflect real usage, including edge cases known to have caused problems before. It should be updated every time a real production failure is diagnosed — the failing case gets added to the dataset so the same regression can never silently reappear.

```
golden_dataset = [
    {"input": "What is your refund policy?",
     "expected_contains": ["30 days", "original payment method"]},
    {"input": "asdkjaslkdj", # edge case: gibberish input
     "expected_behavior": "graceful clarification request, no hallucinated answer"},
]

def run_eval(dataset, generate_fn):
    results = []
    for case in dataset:
        output = generate_fn(case["input"])
        results.append({"input": case["input"], "output": output})
    return results
```

18.5 Chapter Summary

- LLM failures are often fluent and confident, making structured evaluation essential rather than optional.
- Golden datasets, LLM-as-judge, human sampling, and A/B testing each catch different kinds of regressions.
- Production observability (logging, cost tracking, anomaly alerts) should be built in from day one.

Chapter 19

Deployment & DevOps for AI-Powered Applications

19.1 What's Different About Deploying AI Features

Deploying an AI-powered feature layers a few extra concerns on top of standard DevOps: managing API keys across environments, handling variable latency and occasional provider outages gracefully, and versioning prompts the same way you version code.

19.2 A Practical CI/CD Flow

```
# .github/workflows/deploy.yml (simplified)
name: Deploy
on: { push: { branches: [main] } }
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm run test      # includes golden-dataset AI eval tests
      - run: npm run build
      - run: npm run deploy
```

Fig. 19.1 — CI/CD pipeline for an AI-powered feature.

Concern	Practice
Secrets management	Use a secrets manager (Vault, AWS Secrets Manager, or platform env vars) — never plain .env in production
Prompt versioning	Store prompt templates in version control, tagged/reviewed like code, not edited live in a dashboard
Graceful degradation	Design a fallback (cached response, simpler non-AI path) for when the model API is slow or down
Cost monitoring	Set budget alerts per environment; a bug that triggers infinite retries can be expensive fast

19.3 Handling Provider Outages Gracefully

A production AI feature should never hard-fail a user-facing flow just because a model provider is briefly degraded. Practical patterns include: caching recent responses for repeat queries, falling back to a secondary provider via a gateway, queuing non-urgent requests for retry, and always showing the user a clear, honest status rather than a silent hang.

19.4 Chapter Summary

- AI features add prompt versioning, latency handling, and cost monitoring on top of standard DevOps practices.
- Include AI evaluation tests in the same CI pipeline as regular unit/integration tests.
- Always design a graceful degradation path for when the model API is unavailable or slow.

Chapter 20

Fine-Tuning, Evaluation Datasets & Model Customization

20.1 When Fine-Tuning Is (and Isn’t) the Right Tool

Fine-tuning further trains a base model on a specific dataset to specialize its behavior — but it is frequently reached for when a simpler solution (better prompting, RAG, or few-shot examples) would have solved the problem faster and cheaper. Fine-tuning earns its cost when you need a consistent style/format at scale, a narrow domain vocabulary the base model handles poorly, or lower latency from a smaller specialized model replacing a larger general one.

Table 20.1 — Fine-tuning approaches compared

Approach	What It Does	Best For
Full fine-tuning	Updates all model weights on new data	Maximum customization; expensive, needs large datasets
LoRA / parameter-efficient tuning	Trains small adapter layers, leaves base weights frozen	Cheaper, faster, good for narrow specialization
Prompt/instruction tuning	Crafts reusable prompt templates and few-shot examples	Fastest, cheapest; solves most real-world customization needs
RAG (as an alternative)	Retrieves relevant data at query time instead of retraining	Best when the underlying knowledge changes frequently

Best Practice Try prompting and RAG first. Move to fine-tuning only once you’ve hit a concrete, measured limitation that better context or instructions can’t solve — not because fine-tuning sounds more advanced.

20.2 Building a Fine-Tuning Dataset Responsibly

- Curate examples that are genuinely representative of production traffic, not just easy cases.
- Include negative/edge examples so the model learns what *not* to do, not only the happy path.
- Remove PII and sensitive data from training examples before they leave your systems.
- Version the dataset itself, the same way you version code and prompts.

20.3 Chapter Summary

- Fine-tuning is one tool among several (prompting, RAG, fine-tuning) and often the most expensive one to reach for first.
- Parameter-efficient methods like LoRA make specialization cheaper than full fine-tuning in most practical cases.
- A responsibly built, versioned dataset matters as much as the fine-tuning method itself.

Chapter 21

Vision, Voice & Multimodal AI Development

21.1 Beyond Text: Why Multimodal Matters

Modern frontier models increasingly accept and reason over images, audio, and video alongside text, opening up a category of applications that pure text models can't address: reading a screenshot to debug a UI issue, transcribing and summarizing a meeting, or generating an image asset directly from a product description.

Table 21.1 — Multimodal AI capability comparison

Capability	Typical Use Case	Consideration
Vision input (image understanding)	Debugging UI from a screenshot, reading a diagram or handwritten note	Resolution and clarity of the image affect accuracy significantly
Voice/audio transcription	Meeting notes, voice assistants, accessibility features	Background noise and accents affect transcription accuracy

Image generation	Marketing assets, placeholder UI imagery, concept art	Style consistency across multiple generations can be difficult
Video understanding	Summarizing recorded demos, content moderation	Highest compute/latency cost of the multimodal categories

21.2 A Voice Assistant Architecture (Conceptual)

```
Microphone --> [ Speech-to-Text ] --> [ LLM reasoning + tool calls ] --> [ Text-to-Speech ] --> Speaker
```

21.3 Chapter Summary

- Multimodal models extend AI development beyond text into vision, voice, and video understanding.
- Each modality introduces its own accuracy considerations tied to input quality (image clarity, audio noise).
- Voice assistant architectures typically chain speech-to-text, LLM reasoning, and text-to-speech stages.

Part VI — Real-World Projects & Career Prep

Chapter 22

Project 1 — Building a RAG-Powered Support Chatbot

22.1 Project Goal

Build a chatbot that answers questions using your own documentation, rather than generic model knowledge — a common real-world request for internal tools, customer support, and product documentation assistants.

22.2 Architecture

```
docs/*.md --> [ chunk + embed ] --> Vector DB (pgvector)
                                     |
User question --> [ embed query ] --> similarity search --> top chunks
                                     |
                                [ LLM + retrieved context ]
                                     |
                                Answer + sources
```

Fig. 22.1 — Support-bot architecture diagram.

22.3 Folder Structure

```
support-bot/
├── ingest/
│   └── embed_docs.py    # chunks + embeds docs into pgvector
├── api/
│   ├── main.py         # FastAPI app
│   └── rag.py          # retrieval + generation logic
├── frontend/
│   └── ChatWidget.tsx   # React chat UI
└── docker-compose.yml
```

22.4 Core Retrieval Function

```
def answer_question(question: str) -> dict:
    query_vector = embed(question)
    chunks = vector_db.similarity_search(query_vector, k=5)
```

```
context = "\n\n".join(c.text for c in chunks)
prompt = f"Answer using only this context:\n{context}\n\nQuestion: {question}"
response = llm.generate(prompt)
return {"answer": response, "sources": [c.source for c in chunks]}
```

Best Practice Always return sources alongside the answer. It builds user trust and makes wrong answers easy to catch and fix at the source.

22.5 Deployment & Scaling Notes

For a production version, run the embedding/ingest job on a schedule (or trigger it on doc-repo commits) so the vector index stays current, cache frequent queries to reduce latency and cost, and add a “confidence” signal (e.g., top similarity score below a threshold) that routes low-confidence questions to a human rather than guessing.

22.6 Chapter Summary

- A RAG chatbot combines document embedding, vector search, and grounded generation.
- Returning sources alongside answers is both a trust feature and a debugging tool.
- This architecture generalizes to internal wikis, customer support, and product documentation assistants.

Chapter 23

Project 2 — Building an AI Code Reviewer

23.1 Project Goal

Build a GitHub Action that automatically reviews pull requests, flagging potential bugs, style issues, and security concerns as PR comments — a genuinely useful tool for small teams without dedicated senior reviewers on every PR.

23.2 Workflow

1. On every pull request, fetch the diff via the GitHub API.
2. Send the diff to the model with a structured review prompt (bugs, security, style, each in its own section).
3. Parse the structured response into individual inline PR comments.
4. Post comments via the GitHub API, tagged clearly as AI-generated feedback for

human review.

```
REVIEW_PROMPT = '''
Review this diff. Return JSON only, with this shape:
{ "issues": [ { "file": str, "line": int, "severity": "bug|security|style",
  "comment": str } ] }
Do not comment on formatting already handled by a linter.
'''
```

Important Always label AI review comments explicitly as AI-generated. Treat them as a first-pass filter for human reviewers, not a replacement for human sign-off on merges.

23.3 Chapter Summary

- An AI code reviewer fetches PR diffs, requests structured JSON feedback, and posts it as inline comments.
- Structured output (JSON) makes the response reliably parseable into individual comments.
- AI review should augment, not replace, human sign-off before merging.

Chapter 24

Project 3 — A Multi-Agent Research Assistant

24.1 Project Goal

Build a small multi-agent system where a planner agent breaks a research question into sub-tasks, a researcher agent gathers information for each sub-task (via web search), and a writer agent synthesizes findings into a coherent report.

24.2 Agent Roles

Agent	Responsibility	Tools Available
Planner	Breaks the question into 3-5 concrete sub-questions	None — pure reasoning
Researcher	Answers each sub-question using web search	web_search, web_fetch
Writer	Synthesizes all findings into a	None — pure synthesis

24.3 Orchestration Logic

```
sub_questions = planner.run(topic)
findings = [researcher.run(q) for q in sub_questions]
report = writer.run(topic, findings)
```

Best Practice Keep agent responsibilities narrow and non-overlapping. The most common multi-agent failure is two agents with fuzzy, overlapping roles producing contradictory or redundant output.

24.4 Chapter Summary

- A planner/researcher/writer split is a simple, effective pattern for multi-agent research tasks.
- Narrow, non-overlapping agent responsibilities prevent the most common multi-agent failure mode.
- This pattern generalizes to any task requiring decomposition, parallel information-gathering, and synthesis.

Chapter 25

Project 4 — An AI SaaS Starter with Billing & Auth

25.1 Project Goal

Build the scaffolding most real AI-powered SaaS products need before any AI feature matters: authentication, subscription billing, usage metering, and a basic dashboard — the boring 80% that determines whether the AI feature can actually be sold.

25.2 Architecture

```
[ Next.js Frontend ] -- auth --> [ NextAuth / Clerk ]
    |
    v
[ API Routes ] -- billing --> [ Stripe ]
    |
    v
[ AI Feature Layer ] -- usage metering --> [ Postgres: usage_events ]
```


Fig. 25.1 — SaaS starter architecture with auth, billing, and AI layer.

25.3 Usage Metering Example

```
def call_ai_feature(user_id: str, prompt: str) -> str:
    if not has_remaining_quota(user_id):
        raise QuotaExceededError("Upgrade your plan to continue.")
    response = llm.generate(prompt)
    log_usage_event(user_id, tokens=response.usage.total_tokens)
    return response.text
```

Best Practice Meter usage at the token level, not just the request level. AI feature costs scale with tokens, not request count, and a single long request can cost far more than ten short ones.

25.4 Folder Structure

```
ai-saas-starter/
├── app/
│   ├── (auth) /
│   ├── (dashboard) /
│   └── api/
│       ├── billing/
│       └── ai/
├── lib/
│   ├── stripe.ts
│   ├── usage.ts
│   └── llm.ts
└── prisma/
    └── schema.prisma
```

25.5 Chapter Summary

- Most of a successful AI SaaS product is standard SaaS scaffolding: auth, billing, usage metering, dashboard.
- Token-level usage metering is essential once an AI feature has real, paying users.
- Build the boring foundation first; the AI feature is often the smallest part of the total codebase.

Chapter 26

Common Mistakes Across the AI Development Lifecycle

26.1 A Consolidated Mistake Log

Stage	Common Mistake	Fix
Prompting	Vague instructions, no format specified	Apply the four pillars: context, task, constraints, format
Development	Accepting AI code without reading it	Review every diff as if a new hire wrote it
Agentic workflows	Unscoped file/shell permissions	Principle of least privilege, git-branch isolation
RAG systems	No evaluation of retrieval quality separately from generation	Test retrieval and generation independently
Security	Treating retrieved content as trusted instructions	Always delimit and treat external content as data only
Production	No prompt versioning or observability	Version prompts like code; log structured metadata from day one
Fine-tuning	Reaching for fine-tuning before trying prompting/RAG	Exhaust cheaper options first; fine-tune only with a measured gap
Team process	No shared project-instructions file across the team	Maintain and review a CLAUDE.md/AGENTS.md as a living document

26.2 Chapter Summary

- Most costly AI development mistakes repeat across projects: vague prompts, unreviewed code, unscoped permissions.
- A short mistake checklist reviewed before each project phase prevents the majority of avoidable failures.
- Security and evaluation discipline matter more, not less, as agentic capability increases.

Chapter 27

Career Preparation & The Road Ahead

27.1 What Employers Are Actually Looking For

Contrary to the fear that AI coding tools make engineers replaceable, hiring signals have shifted toward judgment: the ability to specify a problem clearly, review AI output critically, and make sound architectural decisions — skills that become more valuable, not less, as raw code generation gets cheaper.

27.2 Interview Questions You Should Be Ready For

Sample Interview Questions 1. Walk me through how you'd review a large AI-generated pull request before merging it. 2. Describe a time an AI tool produced confidently wrong code. How did you catch it? 3. How would you decide between a no-code builder and a custom codebase for a new product? 4. What guardrails would you put around an autonomous coding agent in a CI pipeline? 5. Explain RAG to a non-technical stakeholder in under two minutes.

27.3 Building a Portfolio That Reflects This Era

- Include at least one project showing agentic/tool-use integration, not just chat-based generation.
- Document your prompt iterations for one project — showing the reasoning behind refinement is a strong signal.
- Be ready to explain a mistake an AI tool made on a project and how you caught and fixed it.

27.4 Where This Is Heading

The tools in this handbook will keep changing — model names, product features, and even entire categories will shift within months of publication. What won't change as quickly is the underlying discipline: clear specification, critical review, scoped permissions, and honest evaluation. Learn the durable skills first, and treat specific tool names as details to keep current, not as the foundation itself.

27.5 How to Keep Learning

- Follow primary sources — official docs and changelogs — over secondhand hype.
- Rebuild your mental model of “state of the art” every few months rather than assuming it's static.
- Keep a running personal log of prompts, mistakes, and fixes; it becomes your most valuable reference over time.

27.6 Chapter Summary

- Judgment, review discipline, and clear specification are the skills gaining value, not losing it.
- A strong portfolio in this era shows agentic integration and honest documentation of mistakes caught.
- Interview prep should include explaining AI-related trade-offs as fluently as explaining code itself.

Thank you for reading this handbook. Build something real with it.

Glossary

Term	Definition
Agentic AI	A system where a model plans, acts, observes results, and adjusts across multiple steps toward a goal
Context window	The maximum amount of text a model can process at once, measured in tokens
Embedding	A numeric vector representation of text capturing semantic meaning
Fine-tuning	Further training a pre-trained model on a specific dataset to specialize its behavior
Hallucination	A model generating plausible-sounding but factually incorrect or fabricated output
MCP	Model Context Protocol — a standard for connecting AI applications to external tools and data
Prompt injection	An attack where untrusted input contains hidden instructions that hijack model behavior
RAG	Retrieval-Augmented Generation — grounding model output in retrieved external data
Token	The basic unit of text a model processes, roughly a word or part of a word
Vector database	A database optimized for storing and searching high-dimensional embedding vectors
LoRA	Low-Rank Adaptation — a parameter-efficient fine-tuning technique that trains small adapter layers
Guardrail	A validation or filtering layer designed to keep AI system behavior within safe, intended bounds

References & Further Reading

This handbook was written from a synthesis of official product documentation, engineering blog posts, open-source project documentation, and hands-on production experience across the tools and patterns discussed. For the most current details on any specific tool, always consult that tool's official documentation directly, since this space changes faster than any printed reference can track.

Appendix: Quick-Reference Cheat Sheets

A.1 Prompting Checklist

- Context stated (stack, environment)
- Task stated concretely, not vaguely
- Constraints listed (what NOT to do)
- Output format specified
- Example given if style/format matters

A.2 Agentic Tool Safety Checklist

- Working on a clean git branch, not main
- Permissions scoped to the project directory only
- Tests required to pass before merge
- Budget/step limit set for long-running loops
- No unsupervised access to secrets or production data

A.3 RAG Quality Checklist

- Chunk size tested against real queries, not assumed
- Metadata filtering in place for date/source/permissions
- Retrieval quality evaluated separately from generation quality
- Sources always returned alongside generated answers

A.4 Production Readiness Checklist

- Prompts versioned in source control
- Structured logging in place (model version, latency, token counts)
- Graceful degradation path defined for provider outages
- Budget alerts configured per environment

Index

Agentic AI, 10, 24–26 · Aider, 25 · AI Studio (Google), 20 · Amazon Q Developer, 25 · AutoGen, 29 · Bolt.new, 31 · ChatGPT, 16 · Claude, 18 · Claude Code, 18 · Cline, 23 · ComfyUI, 32 · Context engineering, 12–13 · Copilot (GitHub), 21 · CrewAI, 29 · Cursor, 22 · DeepSeek, 32 · Embeddings, 27 · Evaluation, 34 · Fine-tuning, 20 · Firebase Studio, 31 · Flowise, 29 · Gemini, 20 · Hugging Face, 32 · LangChain, 29 · LlamaIndex, 29 · LM Studio, 32 · Lovable, 31 · MCP (Model Context Protocol), 26 · n8n, 33 · Ollama, 32 · OpenRouter, 32–33 · Perplexity, 33 · Phind, 33 · Prompt engineering, 11 · Prompt injection, 33 · Qwen, 32 · RAG, 27 · Raycast AI, 33 · Replit, 31 · Roo Code, 23 · Semantic Kernel, 29 · Sourcegraph Cody, 25 · v0, 31 · Vector databases, 27 · Vibe coding, 9 · Warp AI, 33 · Windsurf, 23

— *End of Handbook* —